



Kanijia Motorcycles System

Cloud accounting, inventory, sales & purchasing — with enterprise role-based security.

Django REST + React/TypeScript · Bilingual (AR/EN) · Double-entry accounting

1 Overview

Kanijia Motorcycles System is a production-oriented ERP for a motorcycle trading business. It manages **products (inventory), customers, suppliers, sales, and purchases**, and automatically keeps a correct set of **double-entry accounting books** behind the scenes — so the owner gets accurate financial reports without doing manual bookkeeping.

Items & Stock

Every product is a stock item with weighted-average cost and live quantity.

Buy → stock up

Posting a purchase increases inventory & supplier debt automatically.

Sell → stock down

Posting a sale deducts stock, books COGS & revenue, raises customer debt.

Admin-only pricing

Only the Super Admin sets prices; staff sell at official prices.

Bilingual

Full Arabic (RTL) and English (LTR) with a one-click toggle.

Audit trail

Every change is logged with who/when/before/after.

2 Technology Stack

Layer	Technology	Notes
Frontend	React + TypeScript + Vite + Tailwind CSS	Responsive, RTL/LTR, React Query for data

Backend	Django 5 + Django REST Framework	JWT auth (SimpleJWT), role-based permissions
Database	PostgreSQL (prod) / SQLite (local)	Money stored as <code>NUMERIC(19,4)</code> — never floats
Reports	openpyxl (Excel), reportlab (PDF)	Every report exports to PDF & Excel with logo
Auth	JWT access + refresh, bcrypt hashing	Login rate-limited (10/min)

3 Architecture

The golden rule: **no business module touches the ledger directly**. Sales, purchases, and inventory emit a balanced “posting event” to the **Accounting Engine**, which is the only code allowed to write journal entries — atomically.

```

React SPA (browser)
  |
  | HTTPS + JWT
  ▼
Django REST API — Guards: JWT → Role/Permission → Validation → Audit
  |
  | — Business modules: Sales · Purchases · Inventory · Customers · Suppliers · Expenses
  |       |
  |       | emit balanced events
  |       ▼
  | — ACCOUNTING ENGINE (the only writer of the ledger)
  |     • checks debits = credits
  |     • checks period open + postable accounts
  |     • writes JournalEntry + JournalLines in ONE transaction
  |       |
  |       ▼
  PostgreSQL → Reports & Dashboard read from the ledger
  
```

4 Modules

Module	What it does
--------	--------------

Dashboard	KPIs: total sales, purchases, cash/bank, receivables, payables, recent activity.
Inventory	Products (items), three price tiers, stock movements, per-product supplier list, low-stock alerts.
Sales	Create & post sales invoices (auto stock-out + COGS), customer payments, credit notes, quotations, A4 PDF.
Purchases	Purchase invoices (auto stock-in), supplier payments, debit notes.
Customers	Customer master + outstanding balance (computed from documents).
Suppliers	Supplier master + payable balance.
Reports	Trial balance, income statement, balance sheet, customer/supplier debts, inventory valuation.
Admin area	Chart of Accounts, Journal Entries, Daily Expenses, User Management, Audit logs.

5 Roles & Access Control

Backend-enforced role-based access. Frontend hides what a role can't use; the API rejects it regardless.

Capability	Super Admin	Accountant	Sales Employee	Viewer
Set product prices	Yes	No	No	No
View cost / margin	Yes	Yes	Hidden	Hidden
Create & post sales	Yes	Yes	Yes	No
Post journal / accounting	Yes	Yes	No	No
View journal entries	Yes	Yes	No	No
Financial reports	Yes	Yes	No	No
Manage users	Yes	No	No	No
View audit log	Yes	No	No	No

6 Pricing & Financial Security

Three price tiers (Super Admin only)

- **Retail** (مفروق) · **Wholesale** (جملة) · **Bulk** (جملة الجملة)
- Plus **cost price**, **minimum selling price**, and **max discount %** — all admin-only.

The server is the source of truth

When a sale is posted, the backend **ignores any price sent by the client** and loads the official tier price and tax from the product. A tampered request like `{ product:1, quantity:5, unit_price:1 }` is overridden to the real price.

- Sales staff cannot edit, override, or invent prices (custom-priced lines are admin-only).
- Selling below the minimum price is rejected (Super Admin may override).
- Cost prices and margins are stripped from API responses for non-cost roles.

Document locking

- Posted invoices, payments and journal entries are **immutable** — no edits.
- Financial records are **never deleted**: states are Draft → Posted → Cancelled.
- Cancelling a posted invoice **reverses the journal entry and restores stock** — fully auditable.

7 Inventory Flow

Buy 10 @ 80 → Stock = 10, avg cost 80 → Sell 3 @ 120 → Stock = 7 (auto -3) →

COGS 240, Revenue 360

- Valuation method: **weighted-average cost**, recomputed on every receipt.
- Quantities change **only** through purchases, sales, returns, or admin adjustments — each writes a stock-movement record (full stock ledger).

- Overselling is blocked (can't sell more than on-hand).
- Each product tracks which suppliers sell it, with each supplier's cost (printable).

8 Accounting Engine (Double-Entry)

Every transaction produces a balanced journal entry. Examples:

Event	Debit	Credit
Sales invoice	Accounts Receivable	Sales Revenue + VAT Payable
+ stock sold	Cost of Goods Sold	Inventory
Customer payment	Cash / Bank	Accounts Receivable
Purchase invoice	Inventory + VAT Receivable	Accounts Payable
Supplier payment	Accounts Payable	Cash / Bank

The engine rejects any unbalanced entry, any posting into a closed period, and any posting to a non-postable account — and writes everything in a single atomic transaction.

9 Reports

All reports are derived live from the ledger and stock records — never from manually-kept balances. Each exports to **PDF** and **Excel** with the company logo.

Trial Balance

All accounts; debits must equal credits.

Income Statement

Revenue – expenses = net profit/loss.

Balance Sheet

Assets = liabilities + equity.

Customer Debts

Who owes you and how much.

Supplier Debts

Who you owe and how much.

Inventory Valuation

Stock quantity × average cost.

10 Audit & Integrity

- **Audit log** records user, action, entity, before/after values, IP, and timestamp — append-only, admin-view only.
- Customer & supplier balances are always computed from invoices/payments/notes — never editable directly.
- Cash/bank balances are derived from ledger entries.
- Money uses fixed-point decimals (no floating-point rounding errors).

11 Running Locally

Backend

```
cd backend
.venv\Scripts\activate
python manage.py migrate
python manage.py seed_coa
python manage.py seed_roles
python manage.py runserver # http://127.0.0.1:8000
```

Frontend

```
cd frontend
npm install
npm run dev # http://localhost:5173
```

Reset all data to zero (keeps users, accounts, settings)

```
python manage.py reset_data --yes
```

Default admin login: `admin` / `Admin12345` — change it after first sign-in. The Django admin panel is at `http://127.0.0.1:8000/admin/`.

12 API Reference (summary)

Base URL: `/api`. All endpoints require a JWT `Authorization: Bearer <token>` except login.

Area	Endpoint
Auth	<code>POST /api/auth/login/</code> , <code>POST /api/auth/refresh/</code> , <code>GET /api/auth/me/</code>
Products	<code>/api/inventory/products/</code> , <code>/products/{id}/suppliers/</code> , <code>/movements/</code>
Sales	<code>/api/sales/invoices/</code> + <code>/post_invoice/</code> , <code>/cancel/</code> , <code>/pdf/</code>
Purchases	<code>/api/purchases/invoices/</code> + <code>/post_invoice/</code> , <code>/cancel/</code>
Customers / Suppliers	<code>/api/customers/</code> , <code>/api/suppliers/</code> (+ <code>/ledger/</code>)
Reports	<code>/api/reports/<name>/</code> , export: <code>/api/reports/<name>/export/?fmt=pdf excel</code>
Interactive docs	<code>GET /api/docs/</code> (Swagger UI)